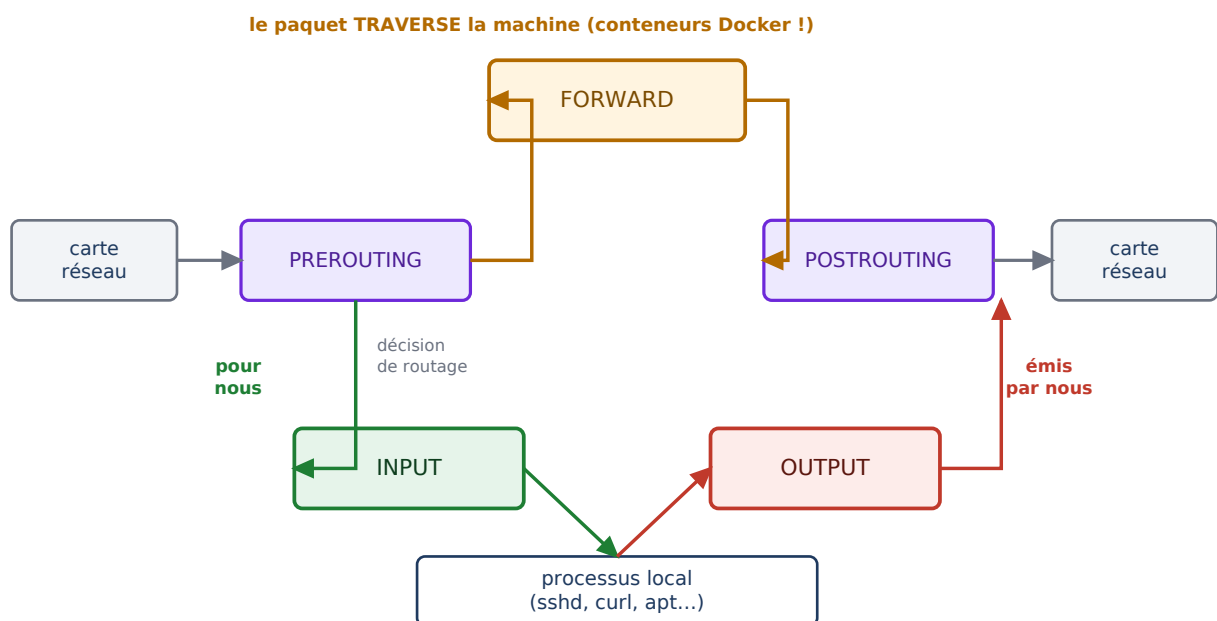


nftables

Filtrer le trafic d'une machine Linux — le cours

Écrit à partir des points bloquants rencontrés pendant le TP « pare-feu d'un honeypot Cowrie ».



Comment lire ce cours. Chaque chapitre part d'une question concrète, explique le mécanisme avec un schéma, puis donne la formulation nftables correspondante. Les encadrés rouges signalent les erreurs classiques — celles qui coûtent une machine ; les encadrés bleus, les réflexes à garder.

Sommaire

	Chapitre	La question à laquelle il répond
1	Filtrer, et pourquoi c'est difficile	Pourquoi une liste de ports ne suffit pas
2	Le voyage d'un paquet	Pourquoi il existe plusieurs chaînes, et laquelle choisir
3	Anatomie d'un ruleset	Que veulent dire <code>table</code> , <code>chain</code> , <code>hook</code> , <code>inet</code>
4	Comment une chaîne est évaluée	Pourquoi l'ordre des règles change tout
5	La <i>policy</i>	La ligne qui transforme une liste en pare-feu
6	Le suivi de connexion	Pourquoi <code>ct state established</code> est partout
7	Source et destination	Pourquoi la même machine est tantôt <code>saddr</code> , tantôt <code>daddr</code>
8	Les flux qu'on oublie toujours	Bouclage, DNS, NTP, ICMP : ce qui casse en silence
9	IPv6	Pourquoi c'est le sujet le plus important d'un honeypot
10	Docker	Pourquoi un conteneur ignore votre chaîne <code>input</code>
11	Écrire, tester, déboguer	La méthode, et comment lire un journal
12	Mémo	Tout tient sur deux pages

1. Filtrer, et pourquoi c'est difficile

L'idée d'un pare-feu paraît simple : décider quels paquets entrent et quels paquets sortent. La difficulté n'est pas d'écrire une règle, elle est ailleurs — dans trois pièges que ce cours va lever un par un.

Piège 1 : une conversation, ce n'est pas un sens unique

Quand ta machine interroge un serveur web, elle envoie des paquets *et* elle en reçoit. Si tu n'autorises que la sortie, rien ne fonctionne : les réponses sont refusées à l'entrée. Si tu ouvres l'entrée en grand pour laisser passer les réponses, tu n'as plus de pare-feu. La solution — le **suivi de connexion** — fait l'objet du chapitre 6.

Piège 2 : tout le trafic ne passe pas au même endroit

Un paquet destiné à ta machine, un paquet qu'elle émet, et un paquet qui la *traverse* pour aller vers un conteneur ne suivent pas le même chemin dans le noyau. Écrire la bonne règle au mauvais endroit, c'est écrire une règle qui ne servira jamais. C'est le chapitre 2.

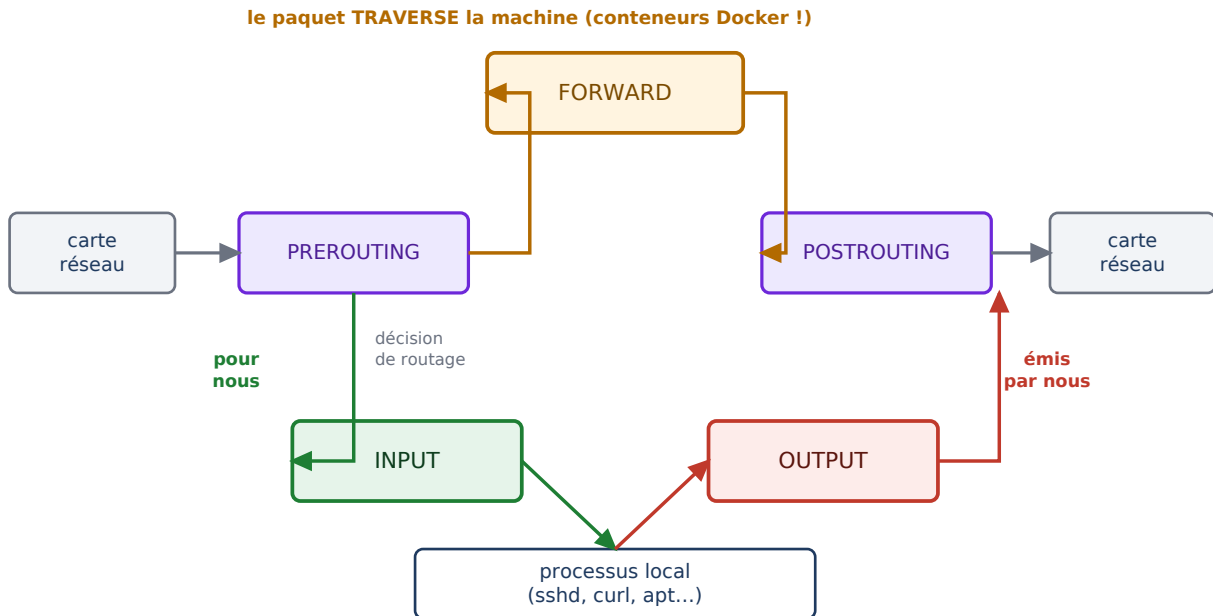
Piège 3 : ce que tu autorises n'est pas ce que tu crois

Une machine moderne émet en permanence du trafic auquel personne ne pense : résolution de noms, synchronisation d'horloge, découverte de voisins IPv6, dialogue avec elle-même. Un pare-feu écrit « proprement » casse presque toujours l'un de ces flux, et le symptôme visible ne désigne jamais la bonne règle. C'est le chapitre 8, et c'est celui qui fait gagner le plus de temps.

Le fil directeur de ce cours : un pare-feu ne s'écrit pas en listant des ports. Il s'écrit en listant des **flux** — qui parle à qui, dans quel sens, et qui ouvre la conversation. Le port n'est que le dernier détail.

2. Le voyage d'un paquet

Le filtrage est intégré au noyau Linux, dans un sous-système appelé **netfilter**. Netfilter place des points de contrôle — des **hooks**, littéralement des crochets — à des endroits précis du parcours d'un paquet. Une chaîne nftables n'est rien d'autre qu'une liste de règles accrochée à l'un de ces points.



Les cinq points de contrôle de netfilter. Un paquet ne passe jamais par les trois chaînes : il suit exactement un des trois chemins.

Les trois chemins possibles

Chemin	Le paquet...	Chaînes traversées	Exemple
Entrant	est destiné à un programme de la machine	prerouting → input	une connexion SSH vers ta machine
Sortant	est émis par un programme de la machine	output → postrouting	curl, l'envoi des logs vers ELK
Traversant	ne fait que passer, il va ailleurs	prerouting → forward → postrouting	tout le trafic d'un conteneur Docker

C'est la décision de routage qui tranche. Juste après *prerouting*, le noyau regarde l'adresse de destination et se demande : « est-ce pour moi ? ». Si oui, le paquet part vers *input*. Sinon, vers *forward*. Tu ne choisis pas : c'est la destination du paquet qui décide, et donc la topologie de ton installation.

La conséquence pratique

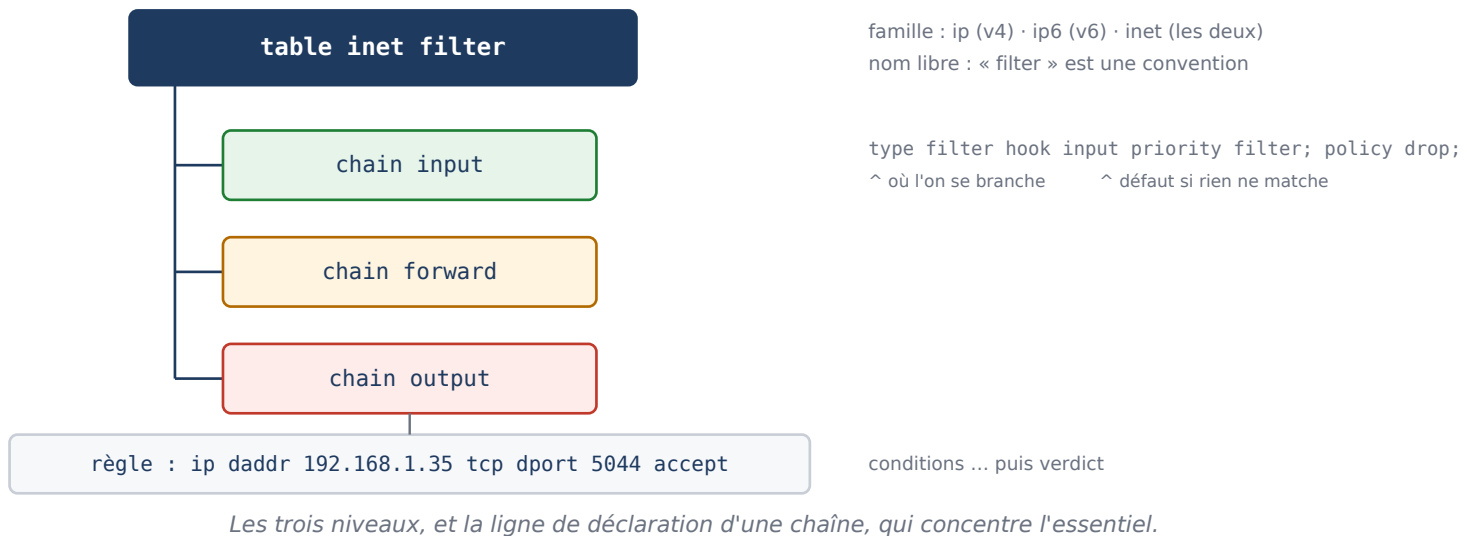
Avant d'écrire une règle, la première question n'est jamais « quel port ? », mais « **ce paquet est-il pour moi, de moi, ou à travers moi ?** ». Trois exemples tirés du TP :

Flux	Qui ouvre la connexion	Chaîne
Ton PC se connecte en SSH à la machine	le PC	input
La machine envoie ses logs à ELK	la machine	output
Un attaquant atteint Cowrie dans un conteneur	l'attaquant, mais la cible est le conteneur	forward

Erreur classique — celle du TP : écrire `tcp dport 22 accept` dans `input` en pensant ouvrir l'accès à Cowrie. Si Cowrie tourne dans un conteneur, ce paquet ne passera jamais par `input`, et la règle ne servira à rien.

3. Anatomie d'un ruleset

Un ruleset s'organise sur trois niveaux : une **table** contient des **chaînes**, qui contiennent des **règles**.



La table et sa famille

La **famille** indique quel type de trafic la table peut voir. C'est le premier mot après table.

Famille	Voit	Remarque
ip	IPv4 seulement	Historique
ip6	IPv6 seulement	Historique — obligeait à tout écrire deux fois
inet	IPv4 et IPv6	Le choix par défaut. Une table, deux protocoles
arp, bridge, netdev	Trafic ARP, ponté, ou au ras de la carte	Cas avancés

Le piège d'inert. La table voit les deux protocoles, mais chaque règle reste spécifique à ce qu'elle nomme :

- `ip saddr 192.168.1.33` → IPv4 uniquement
- `ip6 saddr fe80::/10` → IPv6 uniquement
- `tcp dport 5044` → **les deux**, car aucune version n'est mentionnée

Autrement dit : déclarer une table `inet` ne couvre pas l'IPv6 automatiquement. Il faut y penser règle par règle.

La ligne de déclaration d'une chaîne

```
type filter hook input priority filter; policy drop;
```

```
~~~~~      ~~~~~      ~~~~~      ~~~~
|          |          |          \-- verdict par défaut (chapitre 5)
|          |          |----- ordre vis-à-vis des autres chaînes
|          |----- à quel point de contrôle on s'accroche
\----- ce que la chaîne a le droit de faire
```

Élément	Valeurs	À quoi ça sert
type	filter, nat, route	filter = accepter ou refuser. nat = réécrire des adresses.
hook	input, output, forward, prerouting, postrouting	Le point de contrôle du chapitre 2
priority	un entier, ou un mot-clé	Si deux chaînes visent le même hook, la priorité la plus basse passe en premier . filter vaut 0, raw vaut -300.
policy	accept (défaut) ou drop	Le verdict si aucune règle ne matche. Chapitre 5.

La priorité n'est pas un détail théorique : c'est le mécanisme qui permet de faire passer ses propres règles **avant** celles que Docker installe. Une chaîne forward déclarée avec priority -10 est évaluée avant celle de Docker, qui est à 0.

L'anatomie d'une règle

```
ip daddr 192.168.1.35    tcp dport 5044    ct state new    accept
~~~~~
condition              condition          condition      verdict
```

Toutes les conditions doivent être vraies EN MÊME TEMPS (c'est un ET).
Le verdict est toujours en dernier.

Les verdicts possibles :

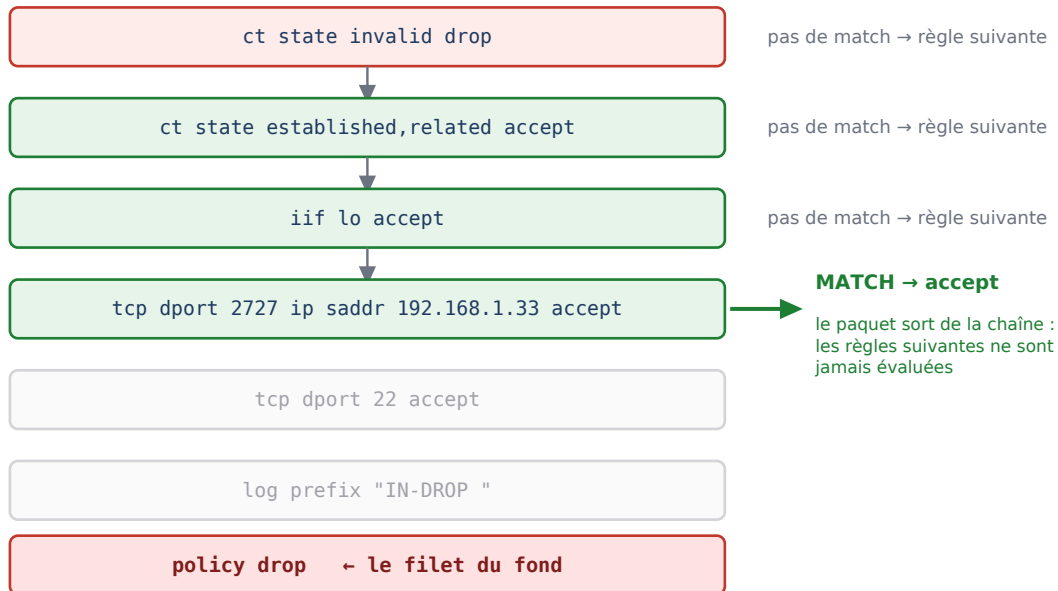
Verdict	Effet
accept	Le paquet est accepté et sort immédiatement de la chaîne
drop	Le paquet est jeté sans un mot. L'émetteur attendra jusqu'à expiration du délai
reject	Le paquet est refusé avec une réponse — l'émetteur sait tout de suite. Plus courtois sur un réseau interne, mais révèle l'existence de la machine
log	N'est pas un verdict. Écrit une ligne dans le journal et l'évaluation continue à la règle suivante
counter	Idem : compte les paquets, n'interrompt rien. Précieux pour déboguer
jump / goto	Renvoie vers une chaîne nommée, pour organiser un gros ruleset

À retenir absolument : log ne bloque rien. Une chaîne qui se termine par log prefix "DROP " sans policy drop écrit « DROP » dans le journal... et laisse passer le paquet. Le préfixe ment. C'est exactement ce qui se produisait sur ta machine.

4. Comment une chaîne est évaluée

Le noyau lit les règles **de haut en bas**, une par une. Dès qu'une règle correspond au paquet et rend un verdict, l'évaluation **s'arrête** : les règles suivantes ne sont jamais examinées.

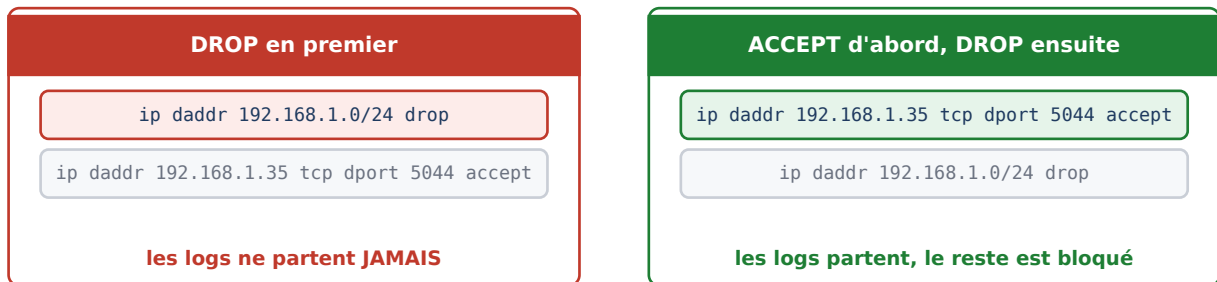
paquet entrant : tcp 192.168.1.33:51234 → 192.168.1.63:2727



Un paquet SSH traverse la chaîne. Il est accepté à la quatrième règle ; les deux suivantes ne le concerneront jamais.

Pourquoi l'ordre change tout

L'ordre compte : la première règle qui matche gagne



Les deux chaînes contiennent exactement les mêmes règles. Seul l'ordre diffère, et le résultat est opposé.

La règle de composition qui en découle, et qu'on retrouve dans tous les pare-feux bien écrits :

1. d'abord ce qui est **anormal** — ct state invalid drop ;
2. puis ce qui est **massif et déjà autorisé** — ct state established,related accept ; c'est par là que passe l'essentiel du trafic, autant l'évacuer tout de suite ;
3. puis le **bouclage** — iif lo accept ;
4. puis les autorisations **du plus précis au plus général** ;
5. enfin la **journalisation**, juste avant la *policy* qui jette le reste.

Effet de bord utile : placer established en deuxième position est aussi un choix de performance. Sur une machine qui reçoit beaucoup de trafic, chaque règle placée au-dessus est évaluée pour chacun des paquets d'une conversation en cours — c'est-à-dire pour la quasi-totalité du trafic.

5. La policy : la ligne qui fait le pare-feu

C'est le point le plus important de tout ce cours. La **policy** est le verdict appliqué à un paquet qu'**aucune règle** n'a reconnu. Elle se déclare sur la ligne de la chaîne, et si tu l'omets, nftables retient accept.

CE QUE TU AS ÉCRIT

... accept

... accept

... accept

log prefix "DROP "

(policy accept implicite)

→ le paquet non prévu PASSE

le log s'affiche... et le paquet passe quand même

CE QU'IL FAUT

... accept

... accept

... accept

log prefix "DROP "

policy drop

→ le paquet non prévu est JETÉ

le log dit ce que tu as oublié d'autoriser

Deux chaînes rigoureusement identiques. Une seule ligne les sépare, et elle décide de tout : à gauche une liste de suggestions, à droite un pare-feu.

Les deux philosophies

	policy accept	policy drop
Principe	Tout passe, sauf ce que j'interdis	Tout est bloqué, sauf ce que j'autorise
Nom courant	liste noire	liste blanche
Ce qu'il faut prévoir	toutes les menaces — impossible	tous les flux légitimes — fini et connaissable
Quand tu oublies quelque chose	une faille , et tu ne t'en aperçois pas	une panne , visible immédiatement et écrite dans le journal
À utiliser	presque jamais	toujours

Le raisonnement qui justifie la liste blanche : dans les deux cas tu vas oublier quelque chose — c'est certain. La seule question est de savoir ce que cet oubli produit. En policy drop, il produit une panne : tu la vois, le journal t'indique le paquet manquant, tu ajoutes la règle, c'est réglé en deux minutes. En policy accept, il produit un trou dont tu n'entendras parler que le jour où quelqu'un l'aura trouvé.

Comment vérifier

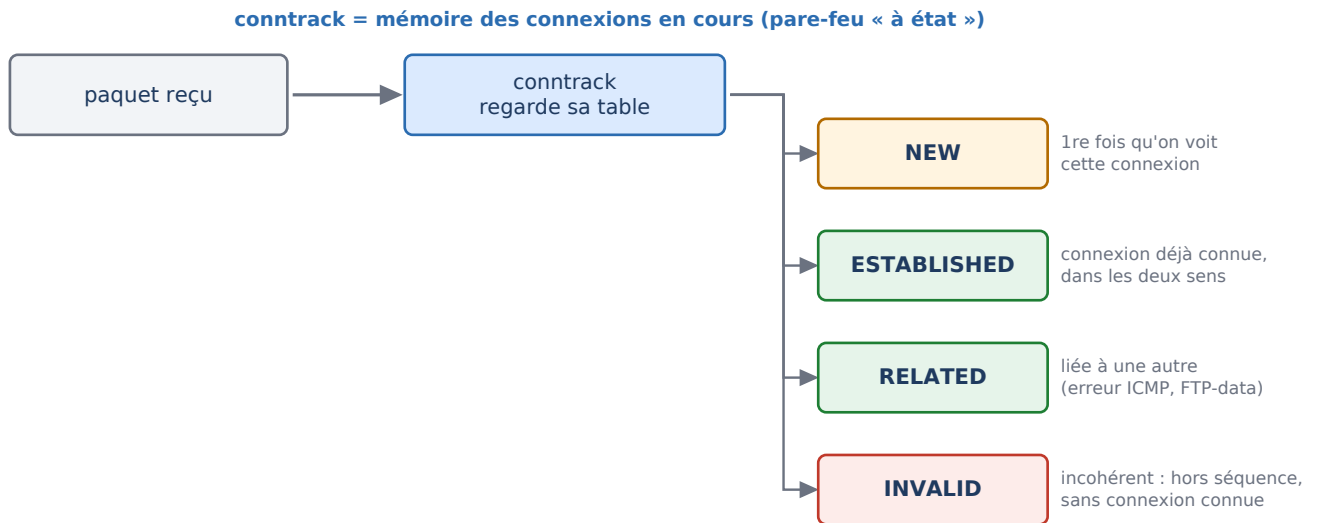
```
sudo nft list ruleset | grep -E 'chain|policy'
```

```
# ce que tu veux lire :  
# type filter hook input priority filter; policy drop;  
# ce qui doit t'alerter :  
# type filter hook input priority filter; policy accept;
```

Note importante : nft **affiche toujours** la policy effective, même quand tu ne l'as pas écrite. C'est donc le moyen le plus fiable de vérifier ce que le noyau a réellement retenu, par opposition à ce que tu crois avoir écrit dans ton fichier.

6. Le suivi de connexion

Le sous-système **conntrack** tient à jour la liste de toutes les conversations réseau en cours. C'est lui qui rend un pare-feu « à état » (*stateful*) — par opposition à un filtre qui examinerait chaque paquet isolément, sans mémoire.



Chaque paquet est confronté à la table des connexions, qui lui attribue un état. C'est cet état que l'on teste avec `ct state`.

Ce que ça change concrètement

Sans suivi de connexion, pour autoriser une simple requête web, il faudrait écrire :

```
# dans output : laisser sortir la requête
tcp dport 443 accept
# dans input : laisser entrer la réponse... mais comment la reconnaître ?
tcp sport 443 accept      # <-- n'importe qui peut émettre depuis le port 443
```

Cette seconde règle est une porte ouverte : elle accepte tout paquet prétendant venir d'un port 443, y compris d'une machine que tu n'as jamais contactée. Avec le suivi de connexion, le problème disparaît :

```
# dans input : une seule règle, pour TOUS les flux, dans les deux sens
ct state established,related accept
```

La réponse à une question posée pendant le TP : non, les paquets de retour ne « contournent » pas le pare-feu. **Chaque paquet est évalué**, sans exception. Simplement, cette règle les reconnaît tous d'un coup, parce que le noyau se souvient que la conversation a été autorisée à s'ouvrir.

Les quatre états, et ce qu'ils signifient

État	Signification	Que faire
new	Premier paquet d'une conversation : quelqu'un demande à ouvrir un dialogue	C'est ici que se prend la décision d'autoriser ou non
established	Paquet appartenant à une conversation déjà connue, dans un sens ou dans l'autre	Accepter — la décision a déjà été prise
related	Paquet lié à une conversation existante sans en faire partie : erreur ICMP, canal de données FTP...	Accepter — sinon certains protocoles cassent de façon très obscure
invalid	Paquet incohérent : hors séquence, appartenant à aucune connexion connue	Jeter en première règle , avant tout le reste

La structure canonique d'une chaîne

```
chain input {
    type filter hook input priority filter; policy drop;

    ct state invalid drop           # 1. le pathologique, tout de suite
    ct state established,related accept # 2. le trafic déjà autorisé (99 % du volume)
    iif lo accept                  # 3. la machine qui se parle à elle-même

    # 4. ... ici, et seulement ici, les ouvertures de connexion autorisées ...

    log prefix "IN-DROP " level info limit rate 5/minute # 5. tracer le reste
}                                                         # 6. policy drop le jette
```

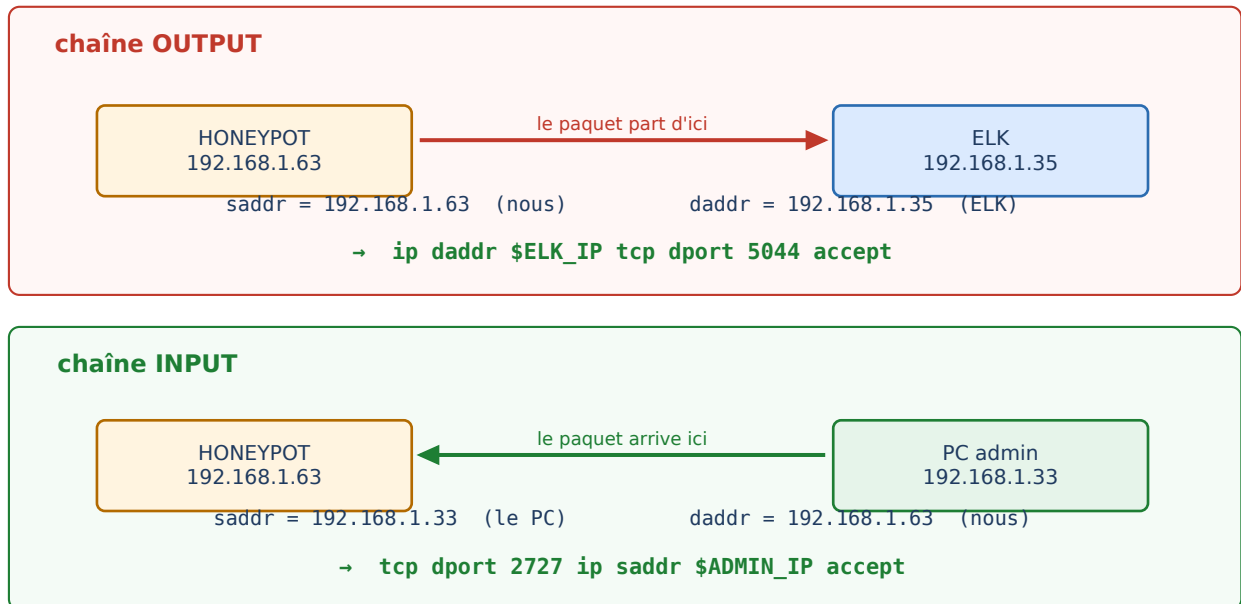
Ces trois premières lignes sont identiques dans toutes les chaînes de tous les pare-feux Linux bien écrits. Apprends-les par cœur : elles constituent le squelette, et la partie 4 est la seule qui change d'une machine à l'autre.

Inspecter les connexions en cours

```
sudo apt install conntrack
sudo conntrack -L           # liste des conversations suivies
sudo conntrack -E           # les voir apparaître en direct
```

7. Source et destination

saddr signifie *source address*, daddr *destination address*. La difficulté n'est pas le vocabulaire : c'est que ces deux valeurs **s'échangent** selon la chaîne, pour une même conversation entre les deux mêmes machines.



La même machine est en daddr quand on lui parle, et en saddr quand elle nous parle. Tout dépend du sens du paquet, jamais du rôle de la machine.

Le tableau de conversion

Dans la chaîne...	saddr désigne	daddr désigne	dport désigne
input	la machine distante	toi	un port ouvert chez toi
output	toi	la machine distante	un port ouvert chez le correspondant

La question qui résout tous les cas : « *qui ouvre la connexion ?* »

- C'est **ma machine** → chaîne output, le correspondant est en daddr, et dport est le port de *son* service.
- C'est **le correspondant** → chaîne input, il est en saddr, et dport est le port de *mon* service.

L'erreur symétrique, dans les deux sens

Écrit	Dans	Ce que ça fait vraiment
<code>ip saddr \$ELK_IP tcp dport 5044 accept</code>	output	Ne matche jamais. En sortie, la source est toujours ta propre adresse. Les logs ne partent pas, et rien n'explique pourquoi.
<code>tcp dport 443 accept</code>	input	Ouvre un service. Ça ne fait pas fonctionner tes requêtes web : ça autorise le monde entier à joindre un serveur web sur ta machine.

Ces deux erreurs ont la même racine : raisonner en « ce flux concerne le port 5044 / le port 443 » plutôt qu'en « qui appelle qui ». La deuxième est la plus dangereuse, parce qu'elle ne provoque aucune panne — elle ouvre juste une porte, silencieusement.

8. Les flux qu'on oublie toujours

Une machine Linux émet en permanence du trafic auquel personne ne pense en écrivant son pare-feu. Voici la liste, par ordre de fréquence des dégâts.

1. Le bouclage — et pourquoi il est vital

```
iif lo accept      # dans input   : "arrivé par l'interface de bouclage"
oif lo accept      # dans output  : "va sortir par l'interface de bouclage"
```

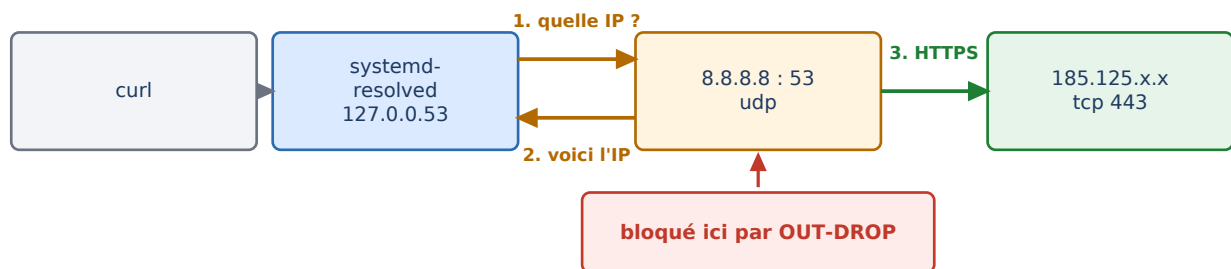
lo est l'interface *loopback* : c'est la machine qui se parle à elle-même, sur 127.0.0.1 et ::1. Ce trafic ne quitte jamais l'ordinateur.

Attention à une confusion fréquente : `iif` se lit *input interface*, et n'a rien à voir avec un *if* conditionnel ni avec une boucle de programmation. `iif lo accept` signifie exactement : « si ce paquet est entré par l'interface de bouclage, accepte-le ». Rien d'autre.

Pourquoi c'est vital sur Ubuntu : `systemd-resolved` écoute sur 127.0.0.53. Chaque résolution de nom passe donc par le bouclage. Sans cette règle et avec `policy drop`, plus rien ne résout, et le symptôme visible sera « Internet ne marche pas ».

2. Le DNS — la première panne de tout pare-feu

Ce qui se passe réellement derrière un simple `curl https://archive.ubuntu.com`



Symptôme observé : « le web ne marche pas » — en réalité l'étape 3 n'est jamais atteinte.

Le paquet bloqué n'est pas du 443 : c'est de l'UDP/53.

Preuve dans tes logs : `OUT-DROP ... PROTO=UDP ... DPT=53 (111 paquets)`

Ce qui se passe réellement derrière une simple requête web. Le port 443 n'est jamais atteint si l'étape 1 est bloquée.

La leçon la plus rentable de ce cours. Quand « le web ne marche plus » après avoir activé un pare-feu, c'est le DNS neuf fois sur dix. Le réflexe correct n'est pas d'ajouter des règles au hasard, c'est de séparer les deux étapes :

```
getent hosts archive.ubuntu.com # teste UNIQUEMENT la résolution de nom
curl -sI https://185.125.190.83 # teste UNIQUEMENT la connexion, sans DNS
```

si le premier échoue et le second passe -> c'est le DNS, pas le web

Et la règle correspondante, en nommant les résolveurs plutôt qu'en ouvrant « le port 53 » :

```
ip daddr { 8.8.8.8, 1.1.1.1 } udp dport 53 accept
ip daddr { 8.8.8.8, 1.1.1.1 } tcp dport 53 accept # TCP : réponses volumineuses, DNSSEC
```

Pour connaître les résolveurs réellement utilisés par ta machine : `resolvectl status`. Attention, il y a souvent un résolveur IPv6 dans la liste, fourni par la box — il faut une règle `ip6 daddr` correspondante.

3. L'horloge

```
udp dport 123 accept      # NTP
```

Sans synchronisation, l'horloge dérive. Sur un honeypot, c'est une perte sèche : des journaux mal horodatés sont inexploitable pour reconstituer une attaque, et ne peuvent plus être corrélés avec ceux des autres machines dans ELK.

4. ICMP — ce n'est pas que le ping

Bloquer tout l'ICMP est une vieille habitude, et une mauvaise idée. Certains messages portent des fonctions indispensables :

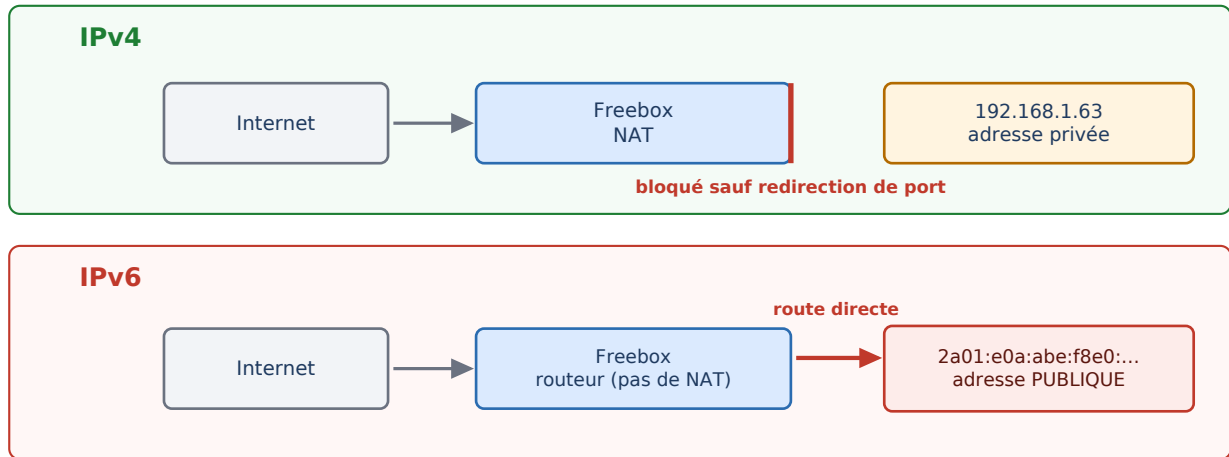
Message	Rôle	Si tu le bloques
echo-request / reply	Le ping	Tu ne peux plus diagnostiquer — gênant, pas grave
destination-unreachable	« Cette adresse n'existe pas / ce port est fermé »	Tes connexions échouent après un long délai au lieu d'échouer tout de suite
time-exceeded	Boucle de routage détectée, base de traceroute	Plus de traceroute
packet-too-big (type 2, IPv6)	Découverte de la taille maximale de paquet	Cassure profonde : les petites connexions passent, les gros transferts se figent au milieu. Diagnostic très difficile

Bonne nouvelle : la plupart de ces messages d'erreur sont classés `related` par le suivi de connexion, donc déjà acceptés par la règle canonique. Il reste à autoriser explicitement `echo-request` si tu veux pouvoir pinger la machine, et **tout l'ICMPv6 de voisinage**, qui n'est lié à aucune connexion (chapitre 9).

9. IPv6

C'est le chapitre le plus important si ta machine est exposée. Deux raisons, indépendantes l'une de l'autre.

Raison 1 : il n'y a pas de NAT en IPv6



En IPv4, la box te protège par effet de bord. En IPv6, elle route — et seul le pare-feu décide.

En IPv4, ta machine porte une adresse privée. Personne, depuis Internet, ne peut l'atteindre sans que tu aies créé une redirection de port sur la box. Cette protection n'est pas voulue : c'est un **effet de bord du manque d'adresses IPv4**.

En IPv6, les adresses ne manquent pas, donc il n'y a pas de NAT : chaque machine reçoit une adresse publique et routable. La box se contente de router. **Le seul rempart devient ton pare-feu.**

```
ip -6 -br addr          # une adresse en 2xxx: est PUBLIQUE et joignable depuis Internet
                        # une adresse en fe80: est locale au lien, elle ne sort pas
```

Conséquence directe pour un honeypot : une machine que tu croyais protégée derrière ta box peut être directement exposée. Et une policy accept oubliée n'est alors plus une négligence théorique — c'est une machine ouverte sur Internet.

Raison 2 : l'IPv6 dépend d'ICMPv6 pour fonctionner

En IPv4, la correspondance entre adresse IP et adresse matérielle se fait par ARP, un protocole distinct qui échappe au filtrage IP. En IPv6, cette fonction est assurée par **ICMPv6** — donc par des paquets que ton pare-feu filtre. Bloquer l'ICMPv6, c'est casser l'IPv6.

Type	Nom	Rôle
135	nd-neighbor-solicit	« qui possède cette adresse ? » — l'équivalent d'ARP
136	nd-neighbor-advert	« c'est moi » — la réponse
133	nd-router-solicit	« y a-t-il un routeur ? »
134	nd-router-advert	La réponse du routeur : préfixe réseau, route par défaut
2	packet-too-big	Taille maximale de paquet — sans lui, les gros transferts se figent

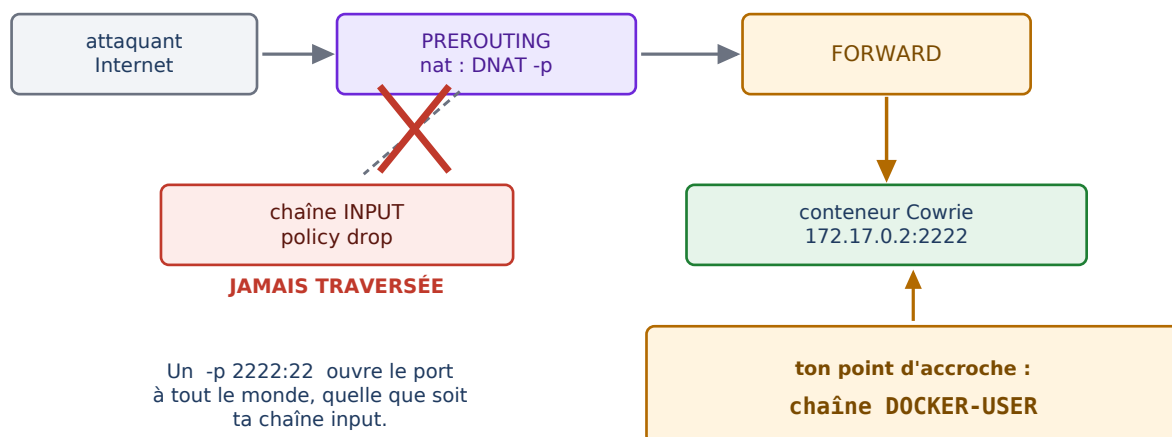
Point qui piège tout le monde : ces messages **ne sont liés à aucune connexion**. La règle `ct state established,related accept` ne les couvre donc pas. Il faut les autoriser explicitement, **et dans les deux chaînes** : une sollicitation de voisin part en output et la réponse arrive en input. Oublier la chaîne output est l'erreur la plus fréquente.

```
# à mettre dans input ET dans output
icmpv6 type { nd-neighbor-solicit, nd-neighbor-advert,
               nd-router-solicit, nd-router-advert,
               packet-too-big, time-exceeded, parameter-problem,
               destination-unreachable } accept
```

Le symptôme typique de l'oubli : le journal du noyau se remplit de centaines de lignes `PROTO=ICMPv6 TYPE=135 refusées`, et l'IPv6 devient intermittent sans raison apparente.

10. Docker

Docker gère lui-même le réseau de ses conteneurs, et il le fait en écrivant ses propres règles de filtrage au démarrage du service. C'est la principale source de surprises.



Règle à retenir : ce qui entre dans un conteneur passe par FORWARD, pas par INPUT.

Le trafic vers un port publié est redirigé avant d'atteindre ta chaîne input, puis routé vers le conteneur en traversant forward.

Ce que Docker fait sans te le dire

- Il active le routage : `net.ipv4.ip_forward=1`.
- Il crée des chaînes à lui : DOCKER, DOCKER-ISOLATION, DOCKER-USER.
- Pour chaque -p, il ajoute une **redirection d'adresse** dans prerouting, c'est-à-dire **avant** la décision de routage.
- Il traduit les adresses sortantes des conteneurs en postrouting.

Les deux conséquences à retenir :

1. Un -p 22:2222 rend le port joignable **quelle que soit ta chaîne** input, parce que le paquet est réécrit puis routé vers le conteneur : il ne passe jamais par input.
2. Le trafic *émis* par un conteneur passe par forward, **pas par** output. Ta chaîne output, si soignée soit-elle, ne filtre pas ce que fait Cowrie.

Le point d'accroche prévu : DOCKER-USER

Docker crée une chaîne vide, DOCKER-USER, évaluée **avant** ses propres règles, et il s'engage à ne jamais y toucher. C'est là que doivent aller tes décisions sur le trafic des conteneurs.

```
# Cowrie a le droit de parler à ELK...
sudo iptables -I DOCKER-USER 1 -d 192.168.1.35 -p tcp --dport 5044 -j ACCEPT
# ...mais à rien d'autre sur le réseau local
sudo iptables -I DOCKER-USER 2 -d 192.168.1.0/24 -j DROP
```

L'ordre est vital, et `-I ... 1` / `-I ... 2` sert précisément à le maîtriser : inversées, ces deux règles bloqueraient aussi les logs.

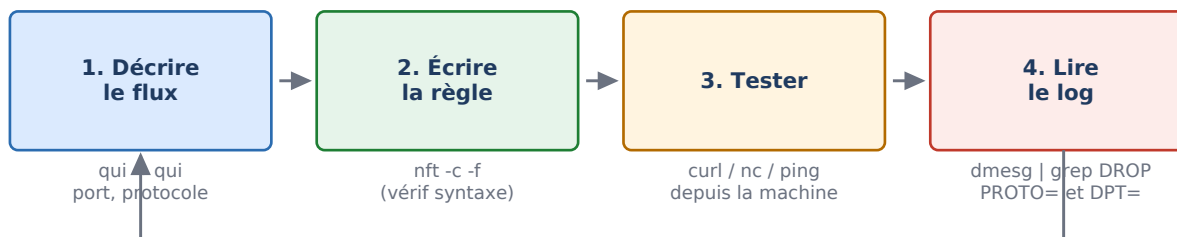
La persistance est le vrai problème. Docker recrée ses chaînes à chaque redémarrage du service, et tes ajouts disparaissent. Trois approches : le paquet `iptables-persistent` ; un `ExecStartPost` ajouté à `docker.service` ; ou — la plus propre — une chaîne `nftables` branchée sur `forward` avec une *priority* inférieure à 0, donc évaluée avant celles de Docker.

L'option radicale

```
/etc/docker/daemon.json  
{ "iptables": false }
```

Docker cesse alors de toucher au pare-feu, et tu écris toi-même les redirections et la traduction d'adresses en nftables. Plus propre et parfaitement lisible — mais tout ce que Docker faisait automatiquement devient ton travail, y compris la sortie Internet des conteneurs. À réserver au moment où le reste est maîtrisé.

11. Écrire, tester, déboguer



le log te dit exactement quel paquet manque : c'est lui qui écrit la règle suivante

La boucle de travail. Le journal n'est pas un constat d'échec : c'est lui qui dicte la règle suivante.

Les trois outils de diagnostic

1. Le journal — ce qui a été refusé

une règle de journalisation, juste avant la fin de chaque chaîne
log prefix "OUT-DROP " level info limit rate 5/minute

```
sudo dmesg | grep OUT-DROP      # relire
sudo journalctl -kf | grep DROP  # suivre en direct
```

Une ligne se lit ainsi :

```
OUT-DROP IN= OUT=wlp3s0 SRC=192.168.1.63 DST=8.8.8.8 PROTO=UDP SPT=51923 DPT=53
```

-----	--	port destination
-----	-----	port source
-----	-----	protocole
-----	-----	vers qui
-----	-----	de qui
-----	-----	interface de sortie
-----	-----	vide = paquet émis par nous

Les deux champs qui résolvent presque tout : PROTO= et DPT=. Ils te disent exactement quel flux manque. Dans l'exemple ci-dessus : UDP vers le port 53, c'est du DNS — il manque la règle DNS. Aucune supposition n'est nécessaire.

Le `limit rate 5/minute` n'est pas décoratif : sans lui, un scan de ports ou une tempête NDP remplit le journal à la vitesse du disque.

2. Les compteurs — ce qui a matché

```
sudo nft list ruleset -a          # -a affiche aussi les "handle"
sudo nft add rule inet filter output ip daddr 192.168.1.35 counter
sudo nft list chain inet filter output      # lire les compteurs
```

Un compteur à zéro sur une règle que tu croyais utilisée est le signe le plus fiable qu'elle ne matche pas. C'est ce test, appliqué à la règle ELK, qui aurait révélé l'inversion `saddr / daddr` en quelques secondes.

3. Les tests, dans le bon ordre

Question	Commande
La route existe-t-elle ?	<code>ip route get 192.168.1.35</code>
La machine répond-elle ?	<code>ping -c2 192.168.1.35</code>
Le port est-il joignable ?	<code>nc -zv 192.168.1.35 5044</code>
La résolution fonctionne-t-elle ?	<code>getent hosts example.com</code>
Que voit-on de l'extérieur ?	<code>nmap -p- 192.168.1.63</code> depuis une autre machine

Le seul test qui prouve quelque chose pour l'accès SSH est une **nouvelle** connexion. Une session déjà ouverte survit grâce à `ct state established`, même si ta règle d'ouverture est fausse. C'est le piège qui coûte des machines : tout semble aller bien jusqu'au moment où l'on se déconnecte.

Le filet de sécurité, en une ligne

```
sudo systemd-run --on-active=180 --unit=fw-rollback nft flush ruleset
# ... appliquer, tester, ouvrir une NOUVELLE session SSH ...
sudo systemctl stop fw-rollback.timer      # annuler le filet, une fois rassuré
```

Le principe : programmer **à l'avance** l'effacement de toutes les règles. Si tu te bloques, tu attends trois minutes et l'accès revient. À préférer à `sleep &` : la tâche est portée par `systemd`, donc elle survit à la perte de ta session — ce qui est exactement le scénario redouté.

12. Mémo

Les commandes

Commande	Effet
<code>nft list ruleset</code>	Affiche toutes les règles actives, policy comprise
<code>nft list ruleset -a</code>	Idem, avec les <i>handle</i> nécessaires pour supprimer une règle
<code>nft -c -f fichier</code>	Vérifie la syntaxe sans rien appliquer
<code>nft -f fichier</code>	Charge le fichier
<code>nft flush ruleset</code>	Efface tout — plus aucun filtrage
<code>nft delete rule inet filter output handle 7</code>	Supprime une règle précise
<code>nft monitor</code>	Affiche en direct les modifications du ruleset (utile avec Docker)
<code>systemctl enable --now nftables</code>	Recharge <code>/etc/nftables.conf</code> au démarrage

Les conditions les plus courantes

Expression	Signification
<code>iif lo / oif lo</code>	Interface d'entrée / de sortie (ici le bouclage)
<code>ip saddr 192.168.1.33</code>	Adresse source IPv4
<code>ip daddr 192.168.1.0/24</code>	Adresse de destination dans un sous-réseau
<code>ip daddr != 192.168.1.0/24</code>	Négation : tout sauf ce sous-réseau
<code>tcp dport { 80, 443 }</code>	Port de destination dans un ensemble
<code>tcp dport 1024-65535</code>	Intervalle de ports
<code>ct state established,related</code>	Connexion déjà connue du suivi
<code>ct state new</code>	Ouverture de connexion — c'est là qu'on décide
<code>icmpv6 type nd-neighbor-solicit</code>	Voisinage IPv6 — indispensable, non couvert par <code>related</code>
<code>meta l4proto</code>	Protocole de transport, indépendamment de la version d'IP
<code>limit rate 5/minute</code>	Limitation de débit, pour les journaux notamment

Les sept erreurs qui coûtent une machine

#	Erreur	Symptôme
1	Oublier <code>policy drop</code>	Aucun. Le pare-feu ne filtre rien et rien ne le signale
2	Oublier <code>established</code> dans <code>output</code>	La session SSH gèle dès l'application des règles
3	Confondre <code>saddr</code> et <code>daddr</code>	La règle ne matche jamais, sans message d'erreur
4	Oublier le DNS	« Internet ne marche plus » — alors que c'est la résolution de noms
5	Oublier <code>iif lo</code>	Le DNS local casse, les services qui se parlent en interne échouent

#	Erreur	Symptôme
6	Oublier l'ICMPv6 en sortie	IPv6 intermittent, journal saturé de types 135/136
7	Oublier <code>systemctl enable nftables</code>	Tout fonctionne... jusqu'au prochain redémarrage

Le squelette à recopier

```
#!/usr/sbin/nft -f
flush ruleset

define ADMIN_IP = 192.168.1.33
define LAN      = 192.168.1.0/24

table inet filter {
    chain input {
        type filter hook input priority filter; policy drop;
        ct state invalid drop
        ct state established,related accept
        iif lo accept
        icmpv6 type { nd-neighbor-solicit, nd-neighbor-advert, nd-router-advert,
                     packet-too-big, time-exceeded, parameter-problem,
                     destination-unreachable, echo-request } accept
        ip saddr $LAN icmp type echo-request accept
        # --- ouvertures autorisées ---
        log prefix "IN-DROP " level info limit rate 5/minute
    }
    chain forward {
        type filter hook forward priority filter; policy drop;
        ct state invalid drop
        ct state established,related accept
        log prefix "FWD-DROP " level info limit rate 5/minute
    }
    chain output {
        type filter hook output priority filter; policy drop;
        ct state invalid drop
        ct state established,related accept
        oif lo accept
        icmpv6 type { nd-neighbor-solicit, nd-neighbor-advert, nd-router-solicit,
                     packet-too-big, time-exceeded, parameter-problem,
                     destination-unreachable } accept
        # --- sorties autorisées ---
        log prefix "OUT-DROP " level info limit rate 5/minute
    }
}
```

La seule chose à retenir si tu ne devais en retenir qu'une. Un pare-feu ne se relit pas, il se teste. Écris la règle, applique, provoque le flux, lis le journal. Le journal te dira toujours ce qui manque — et il ne se trompe jamais, contrairement au raisonnement qu'on tient devant son fichier.